

# Tyflow Studio 研发质量闭环建设汇报

从需求理解到开发测试交付的可追踪智能研发流水线

阶段成果汇报 2026 年 9 月

## 一 汇报结论

**当前结论** Tyflow Studio 已完成核心体系搭建，具备真实需求试运行条件。

该体系将需求录入、计划确认、受控开发、专业测试、人工验收和代码提交串成一条可追踪流程，并通过三道硬闸降低 AI 开发中最常见的理解偏差、范围漂移和测试失真风险。当前版本已通过完整自动化回归，可选择代表性项目进入试点，以真实交付数据验证效率和质量收益。

| 判断维度   | 当前状态                      | 管理结论    |
|--------|---------------------------|---------|
| 流程完整度  | 需求到提交的主链路已经闭环             | 具备试运行条件 |
| 开发防偏   | 需求 计划 文件范围 测试用例显式追踪       | 核心约束已建立 |
| 测试可信度  | P0 P1 强制结构化证据并绑定代码快照      | 可审计 可复核 |
| 独立使用能力 | Tyflow 无 Studio 时保留完整原生验证 | 能力不降级   |
| 工程稳定性  | 10 个测试套件通过 102 项通过 1 项跳过  | 当前回归全绿  |

## 二 背景与需要解决的问题

AI 可以明显提升前端研发速度，但仅提高代码生成速度并不能直接转化为稳定交付。实际风险集中在三个环节。

| 风险 | 典型表现 | 业务影响 |
|----|------|------|
|----|------|------|

|        |                         |                   |
|--------|-------------------------|-------------------|
| 理解跑偏   | 需求被错误拆解 计划和验收标准不一致      | 返工增加 交付结果偏离业务目标   |
| 开发跑偏   | 实际修改超出计划 顺手改动路由 权限或依赖   | 影响面扩大 回归成本和上线风险增加 |
| 测试跑偏   | 测试只验证实现本身 只有文字结论 缺少真实证据 | 测试通过但业务仍可能错误      |
| 重复消耗   | 开发代理和测试代理重复执行相同检查       | Token 时间和算力成本增加   |
| 过程不可追溯 | 无法回答基于哪个需求 哪版代码 哪次测试放行  | 评审困难 经验无法沉淀       |

### 三 总体方案

方案采用分层职责设计。Tyflow 管理需求规格和开发约束，专业测试 Skill 负责测试设计 执行和证据，Studio 负责流程编排 状态持久化 结果复用和强制门禁。三者通过结构化契约连接，既可以组合运行，也可以独立使用。

**需求录入 → 只读评估 → 计划确认 → 受控开发 → 专业测试 → 人工验收 → 提交推送**

| 层级                            | 主要职责                   | 设计原则        |
|-------------------------------|------------------------|-------------|
| Tyflow                        | 需求规格 计划 测试意图 追踪关系 独立验证 | 单独使用时能力不降级  |
| frontend test 或 test engineer | 测试计划 执行 证据 结论          | 专业工具负责专业测试  |
| Studio                        | 编排 快照 门禁 复用 审计 启动与提交   | 用机器规则阻止流程绕过 |

## 四 三道硬闸

### 第一道 需求计划测试追踪闸

每条核心需求必须被明确的计划步骤覆盖，每个计划步骤必须声明预计修改文件，每条 P0 P1 测试用例必须指向需求和计划步骤。旧任务可以读取，但不能依靠自动补关系通过新门禁。

### 第二道 测试证据闸

P0 P1 用例只有文字描述不能判定通过。命令证据必须包含真实命令和零退出码，文件证据必须存在于受控目录，必要时校验内容哈希。失败 阻塞 未执行和环境缺失都会被如实保留。

### 第三道 代码差异闸

计划步骤通过 expectedFiles 声明预计文件范围，Studio 将实际 Git 改动逐文件匹配到计划 需求和测试。计划外文件 缺少测试覆盖或高风险改动会阻止启动 验收或提交。测试执行前后还会比较内容快照，禁止测试代理在验证阶段修改业务代码后直接放行。

## 五 已形成的关键能力

- 需求 计划 文件 测试 证据五级追踪，可定位每个结论的来源。
- 冻结分支 提交 计划和代码快照，避免评估对象和执行对象不一致。
- 开发阶段与正式测试阶段分离，减少重复测试和 Token 消耗。
- 代码 计划 测试 Skill 和证据完全一致时复用结果，任一变化自动失效。
- TestRequest V1 和 TestResult V1 提供机器可校验的测试协议。
- 测试阶段只允许修改测试资产，业务和配置变化必须回到开发审查。
- Tyflow 支持 standalone 和 orchestrated 两种模式，独立使用仍执行完整验证。
- 测试失败 修复 重测 启动 人工验收 提交推送形成连续状态链。

## 六 预期价值

| 价值方向     | 作用机制                  | 建议试点指标            |
|----------|-----------------------|-------------------|
| 减少返工     | 开发前暴露理解偏差和验收缺口        | 计划纠正次数 需求返工次数     |
| 降低变更风险   | 实际改动必须落在计划文件范围内       | 计划外改动拦截次数         |
| 提高测试可信度  | 核心用例必须有可复核证据          | 证据完整率 人工抽查通过率     |
| 控制 AI 成本 | 正式测试只执行一次 可安全复用       | 重复命令减少量 Token 和耗时 |
| 提升审计能力   | 结果绑定代码快照 计划和 Skill 版本 | 问题定位时间 交付追溯完整率    |
| 沉淀团队能力   | 规则 错题和验证资产持续复用        | 组件复用率 同类问题复发率     |

## 七 当前成熟度与边界

当前版本适合作为 V1 进入真实项目试运行。主流程和硬闸已经完成，但暂不建议直接作为无人值守发布系统。人工仍需确认业务语义 视觉体验 高风险权限和最终验收。

| 类别   | 当前判断             | 说明                   |
|------|------------------|----------------------|
| 已具备  | 单机真实需求闭环         | 可完成计划 开发 测试 启动 验收和提交 |
| 已具备  | 严格门禁和证据链         | 核心结论可追踪 可抽查          |
| 保留人工 | 业务语义和视觉体验        | AI 难以完全自证            |
| 后续增强 | 部分测试结果复用         | 当前支持完全匹配复用           |
| 后续增强 | Worktree 隔离和并行任务 | 提升多任务并行安全性           |
| 后续增强 | 团队审批和审计视图        | 适用于多人协同和规模化使用        |

## 八 建议试运行方案

建议先以三类任务进行四周试运行，不急于继续扩大功能范围。通过真实失败和真实交付数据决定 V1.1 优先级。

| 试点任务       | 验证重点            | 通过标准             |
|------------|-----------------|------------------|
| 小型 Bug     | 快速修复路径和基础回归     | 流程成本可控 且原问题有证据闭环 |
| 标准 L2 页面需求 | 完整计划 追踪 差异和测试流程 | 核心需求无断链 计划外改动可识别 |
| 接口 权限或路由需求 | 高风险差异和真实行为验证    | 关键风险被门禁或人工审批覆盖   |

试运行建议记录：需求总耗时 人工确认次数 计划外改动拦截数 测试重复减少量 结果复用次数 Token 消耗 误报 漏报，以及最终人工发现但系统未发现的问题。

## 九 建议决策与所需支持

建议批准 Tyflow Studio 进入受控试运行，并指定 1 至 2 个前端项目和三类代表性需求。试运行阶段不改变现有发布审批制度，Studio 作为研发和提测前的质量增强层运行。

- 确定试点项目和需求负责人。
- 允许在真实需求中记录效率 质量和 Token 数据。
- 安排研发和测试各一名代表参与结果抽查。
- 四周后基于数据评审是否扩大项目范围，并确定 V1.1 投入。

## 附录 当前验证结果

本轮完成 TypeScript ESLint JSON Schema 和完整自动化测试验证。

| 检查项         | 结果                  |
|-------------|---------------------|
| TypeScript  | 通过                  |
| ESLint      | 通过                  |
| JSON Schema | 解析通过                |
| 测试套件        | 10 个通过              |
| 测试用例        | 102 个通过 1 个跳过 0 个失败 |